

Module 3 React Fundamentals

PROF. SUNITA D RATHOD

React Js- Life cycle of component

The life cycle of a React component can be divided into three main phases: **Mounting, Updating, and Unmounting**. Here's a breakdown of each phase along with the important methods involved:

1. Mounting

This phase includes the creation and insertion of a component into the DOM.

- **constructor(props)**: This method is called when the component is initialized. It's used to set up the initial state and bind methods.

- **static getDerivedStateFromProps(props, state):** This method is called right before rendering. It allows the state to be updated based on the props.
- **render():** This method is required in every class component. It returns the JSX that defines the component's UI.
- **componentDidMount():** This method is called immediately after the component is inserted into the DOM. It is typically used for side effects like fetching data.

2. Updating

This phase occurs when a component is re-rendered due to changes in props or state.

- **static getDerivedStateFromProps(props, state):** This method is called right before rendering when new props or state are received. It can return an object to update the state.
- **shouldComponentUpdate(nextProps, nextState):** This method determines whether the component should be re-rendered. It can be used to optimize performance by preventing unnecessary renders.
- **render():** This method is called to re-render the component.

- **getSnapshotBeforeUpdate(prevProps, prevState)**: This method is called right before the changes from the virtual DOM are to be reflected in the real DOM. It can capture information from the DOM before it is potentially changed.
- **componentDidUpdate(prevProps, prevState, snapshot)**: This method is called immediately after the update is reflected in the DOM. It's a good place to perform operations based on the updated DOM.

3. Unmounting

This phase involves the removal of a component from the DOM.

- **componentWillUnmount()**: This method is called right before the component is removed from the DOM. It is used for cleanup tasks like invalidating timers, canceling network requests, or cleaning up subscriptions.

Initialization

Set Props and
Initial State of
the component
in the constructor

Mounting

render()

componentDidMount()

Updation

For Props
getDerivedStateFromProps()

For State
setState()

shouldComponentUpdate()

getSnapshotBeforeUpdate()

render()

ComponentDidUpdate()

Unmounting

ComponentWillUnmount()

React Methods



ReactDOM Methods



Other Lifecycle Methods

There are a few other methods related to error handling:

- **static `getDerivedStateFromError(error)`:** This method is called when a descendant component throws an error. It allows updating the state to display an error boundary.
- **`componentDidCatch(error, info)`:** This method is called after an error has been thrown by a descendant component. It can be used to log the error or display a fallback UI.

React Events

Just like HTML DOM events, React can perform actions based on user events.

React has the same events as HTML: click, change, mouseover etc.

Adding Event

React events are written in camelCase syntax:

`onClick` instead of `onclick`.

React event handlers are written inside curly braces:

`onClick={shoot}` instead of `onclick="shoot()"`.

Events

React:

```
<button onClick={shoot}>Take the Shot!</button>
```

HTML:

```
<button onclick="shoot()">Take the Shot!</button>
```

Example:

Put the `shoot` function inside the `Football` component:

```
function Football() {  
  const shoot = () => {  
    alert("Great Shot!");  
  }  
  
  return (  
    <button onClick={shoot}>Take the shot!</button>  
  );  
}  
  
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(<Football />);
```

Passing Arguments

To pass an argument to an event handler, use an arrow function.

```
function Football() {  
  const shoot = (a) => {  
    alert(a);  
  }  
  
  return (  
    <button onClick={() => shoot("Goal!")}>Take the shot!</button>  
  );  
}  
  
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(<Football />);
```

React

Event Object

Event handlers have access to the React event that triggered the function.

```
function Football() {  
  const shoot = (a, b) => {  
    alert(b.type);  
    /*  
     'b' represents the React event that triggered the function,  
     in this case the 'click' event  
    */  
  }  
  
  return (  
    <button onClick={(event) => shoot("Goal!", event)}>Take the shot!</button>  
  );  
}  
  
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(<Football />);
```

FORMS

Just like in HTML, React uses forms to allow users to interact with the web page.

Example:

Add a form that allows users to enter their name:

```
function MyForm() {  
  return (  
    <form>  
      <label>Enter your name:  
        <input type="text" />  
      </label>  
    </form>  
  )  
}  
  
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(<MyForm />);
```

Handling FORMS

In HTML, form data is usually handled by the DOM.

In React, form data is usually handled by the components.

When the data is handled by the components, all the data is stored in the component state.

You can control changes by adding event handlers in the **onChange** attribute.

We can use the **useState** Hook to keep track of each inputs value and provide a "single source of truth" for the entire application.

FORMS

Example:

```
import { useState } from 'react';
import ReactDOM from 'react-dom/client';

function MyForm() {
  const [name, setName] = useState("");

  return (
    <form>
      <label>Enter your name:
      <input
        type="text"
        value={name}
      />
    </form>
  );
}
```

FORMS

```
      onChange={(e) => setName(e.target.value)}  
    />  
  </label>  
</form>  
)  
}
```

```
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(<MyForm />);
```

Submitting Forms

```
import { useState } from "react";
import ReactDOM from 'react-dom/client';

function MyForm() {
  const [name, setName] = useState("");

  const handleSubmit = (event) => {
    event.preventDefault();
    alert(`The name you entered was: ${name}`);
  }
}
```

```
return (  
  <form onSubmit={handleSubmit}>  
    <label>Enter your name:  
      <input  
        type="text"  
        value={name}  
        onChange={(e) => setName(e.target.value)}  
      />  
    </label>  
    <input type="submit" />  
  </form>  
)  
}
```

```
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(<MyForm />);
```

Enter your name:

the name you entered was: LUCKY

Controlled Components

In HTML, form elements such as `<input>`, `<textarea>`, and `<select>` typically maintain their own state and update it based on user input.

In React, mutable state is typically kept in the state property of components, and only updated with `setState()`.

An input form element whose value is controlled by React in this way is called a “controlled component”.

```
class NameForm extends React.Component {  
  constructor(props) {  
    super(props);  
    this.state = {value: ''};  
  
    this.handleChange = this.handleChange.bind(this);  
    this.handleSubmit = this.handleSubmit.bind(this);  
  }  
  
  handleChange(event) {  
    this.setState({value: event.target.value});  
  }  
  
  handleSubmit(event) {  
    alert('A name was submitted: ' + this.state.value);  
    event.preventDefault();  
  }  
}
```

```
render() {  
  return (  
    <form onSubmit={this.handleSubmit}>  
      <label>  
        Name:  
        <input type="text" value={this.state.value} onChange={this.handleChange} />  
      </label>  
      <input type="submit" value="Submit" />  
    </form>  
  );  
}
```


Name:

An embedded page at [cdpn.io](#) says

A name was submitted: lucky

OK

SINGLE PAGE APPLICATION

A Single Page Application (SPA) is a type of web application that interacts with the user by dynamically rewriting the current page, rather than loading entire new pages from the server.

This approach makes the application feel faster and more responsive because it doesn't need to reload the entire page.

SINGLE PAGE APPLICATION USING REACT ROUTER

- React Router is a library for routing in React applications. It enables navigation between different components in a single-page application (SPA) without refreshing the page.
- It also allows for dynamic routing, nested routes, and more.
- **Installation**

To use React Router, you first need to install it:

npm install react-router-dom

Basic Setup

Here is a basic setup for React Router in a React application:

Import BrowserRouter, Route, and Switch:

```
import React from 'react';
import { BrowserRouter as Router, Route, Switch } from 'react-router-dom';
import Home from './Home';
import About from './About';
import Contact from './Contact';

const App = () => (
  <Router>
    <Switch>
      <Route exact path="/" component={Home} />
      <Route path="/about" component={About} />
      <Route path="/contact" component={Contact} />
    </Switch>
  </Router>
);

export default App;
```

Core Components

1. **BrowserRouter (Router):**

- This is the main component that wraps your entire application, enabling the use of React Router.

Route:

- This component is used to define a path and the component that should be rendered when the path matches the URL.

Switch:

- This component renders the first Route that matches the current location. It is useful for ensuring only one route is rendered at a time.

Link:

- This component is used for navigation between routes. It renders an anchor (<a>) element and prevents a full page reload when the link is clicked.

```
import { Link } from 'react-router-dom';

const Navbar = () => (
  <nav>
    <ul>
      <li><Link to="/">Home</Link></li>
      <li><Link to="/about">About</Link></li>
      <li><Link to="/contact">Contact</Link></li>
    </ul>
  </nav>
);
```

Styling React Using CSS

There are many ways to style React with CSS, this tutorial will take a closer look at three common ways:

- Inline styling
- CSS stylesheets
- CSS Modules

Inline Styling

To style an element with the inline style attribute, the value must be a JavaScript object:

Hello Style!

Add a little style!

```
const Header = () => {  
  return (  
    <>  
      <h1 style={{color: "red"}}>Hello Style!</h1>  
      <p>Add a little style!</p>  
    </>  
  );  
}
```

camelCased Property Names

Since the inline CSS is written in a JavaScript object, properties with hyphen separators, like `background-color`, must be written with camel case syntax:

```
<h1 style={{backgroundColor: "lightblue"}}>Hello
```

```
Style!</h1>
```

JavaScript Object

You can also create an object with styling information, and refer to it in the style attribute:

Hello Style!

Add a little style!

```
const Header = () => {  
  const myStyle = {  
    color: "white",  
    backgroundColor: "DodgerBlue",  
    padding: "10px",  
    fontFamily: "Sans-Serif"  
  };  
  return (  
    <>  
      <h1 style={myStyle}>Hello Style!</h1>  
      <p>Add a little style!</p>  
    </>  
  );  
}
```

CSS Stylesheet

You can write your CSS styling in a separate file, just save the file with the `.css` file extension, and import it in your application.

```
import './App.css';
```

App.css:

Create a new file called "App.css" and insert some CSS code in it:

```
body {  
  background-color: #282c34;  
  color: white;  
  padding: 40px;  
  font-family: Sans-Serif;  
  text-align: center;  
}
```

Hello Style!

Add a little style!

CSS Modules

Another way of adding styles to your application is to use CSS Modules.

CSS Modules are convenient for components that are placed in separate files.

Create the CSS module with the `.module.css` extension, example:
`my-style.module.css`.

my-style.module.css:

```
.bigblue {  
  color: DodgerBlue;  
  padding: 40px;  
  font-family: Sans-Serif;  
  text-align: center;  
}
```

Car.js:

```
import styles from './my-style.module.css';

const Car = () => {
  return <h1 className={styles.bigblue}>Hello Car!</h1>;
}

export default Car;
```

index.js:

Hello Car!

```
import ReactDOM from 'react-dom/client';  
import Car from './Car.js';  
  
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(<Car />);
```


REACT JS ANIMATION

The animation is a technique in which images are manipulated to appear as moving images.

we can add animation using an explicit group of components known as the **React Transition Group**.

React Transition Group is an add-on component for managing component states and useful for defining **entering** and **exiting** transitions.

React Animation

React Transition group has mainly **two APIs** to create transitions. These are:

1. **ReactTransitionGroup:** It uses as a low-level API for animation.
2. **ReactCSSTransitionGroup:** It uses as a high-level API for implementing basic CSS transitions and animations.

React Animation

We need to install **react-transition-group** for creating animation in React Web application. You can use the below command.

```
$ npm install react-transition-group --save
```

React Animation

React Transition Group API provides **three** main components. These are:

1. Transition
2. CSSTransition
3. Transition Group

React Animation

Transition

It has a simple component API to describe a transition from one component state to another over time. It is mainly used to animate the **mounting** and **unmounting** of a component. It can also be used for in-place transition states as well.

We can access the Transition component into four states:

entering

entered

exiting

exited

React Animation

CSSTransition

The CSSTransition component uses CSS stylesheet classes to write the transition and create animations. It is inspired by the **ng-animate** library. It can also inherit all the props of the transition component. We can divide the "CSSTransition" into **three** states. These are:

Appear

Enter

Exit

React Animation

CSSTransition component must be applied in a pair of class names to the child components. The first class is in the form of **name-stage** and the second class is in the **name-stage-active**.

For example, you provide the name fade, and when it applies to the 'enter' stage, the two classes will be **fade-enter** and **fade-enter-active**. It may also take a prop as Timeout which defines the maximum time to animate.

React Animation

TransitionGroup

This component is used to manage a set of transition components (Transition and CSSTransition) in a list. It is a state machine that controls the **mounting** and **unmounting** of components over time. The Transition component does not define any animation directly. Here, how 'list' item animates is based on the individual transition component. It means, the "TransitionGroup" component can have different animation within a component.

React Animation

we will create a basic React component.

The **ReactCSSTransitionGroup** element will be used as a wrapper of the component we want to animate.

It will use **transitionAppear** and **transitionAppearTimeout**, while **transitionEnter** and **transitionLeave** are false.

App.jsx

```
import React from 'react';
var ReactCSSTransitionGroup = require('react-addons-css-transition-group');

class App extends React.Component {
  render() {
    return (
      <div>
        <ReactCSSTransitionGroup transitionName = "example"
          transitionAppear = {true} transitionAppearTimeout = {500}
          transitionEnter = {false} transitionLeave = {false}>

          <h1>My Element...</h1>
        </ReactCSSTransitionGroup>
      </div>
    );
  }
}

export default App;
```

main.js

```
import React from 'react'
import ReactDOM from 'react-dom';
import App from './App.jsx';

ReactDOM.render(<App />, document.getElementById('app'));
```

The CSS animation is very simple.

css/style.css

```
.example-appear {
  opacity: 0.04;
}
.example-appear.example-appear-active {
  opacity: 2;
  transition: opacity 50s ease-in;
}
```

React Animation

Once we start the app, the element will fade in.



Thank you